HMWK 2

From Wallet Seed to Ethereum Address

Hashing, Key Derivation and Cryptographic Primitives

Course: Blockchain & Distributed Ledger Technologies

Sapienza Università di Roma

October 13, 2026

Contents

1 Overview: The Ethereum Wallet Pipeline **3**
- 1.1 The pipeline at a glance . . . 3
- 1.2 Five fundamental operations – a preliminary taxonomy . . . 3

2 Temporal and Historical Context **4**
- 2.1 SHA-2 vs. Keccak-256 / SHA-3: a subtle but important distinction . . . 4

3 BIP-39 — Mnemonic Sentence and Seed Generation **5**
- 3.1 Purpose and role . . . 5
- 3.2 Entropy generation . . . 5
- 3.3 Seed derivation: PBKDF2-HMAC-SHA512 . . . 6
- 3.4 Key properties . . . 6

4 BIP-32 — Hierarchical Deterministic Key Derivation **6**
- 4.1 Motivation . . . 6
- 4.2 The master node . . . 6
- 4.3 Child key derivation . . . 7
- 4.4 Extended keys and serialisation . . . 7

5 BIP-44 — Standard Derivation Paths for HD Wallets **7**
- 5.1 The problem: path chaos . . . 7
- 5.2 The BIP-44 path . . . 7
- 5.3 Why hardened levels matter here . . . 8

6 secp256k1 — Elliptic-Curve Public-Key Cryptography **8**
- 6.1 The curve . . . 8
- 6.2 From private key to public key . . . 9
- 6.3 Public key format . . . 9
- 6.4 Why secp256k1 and not RSA? . . . 9

7 Keccak-256 — Hash Function for Address Derivation **9**
- 7.1 From public key to address . . . 9
- 7.2 Sponge construction . . . 10
- 7.3 Required properties and why they matter . . . 10
- 7.4 Statistical properties of Keccak-256 outputs . . . 10

8 EIP-55 — Mixed-Case Checksum Encoding **11**
- 8.1 The problem . . . 11
- 8.2 The EIP-55 algorithm . . . 11
- 8.3 Error detection capability . . . 12
- 8.4 Backward compatibility . . . 12

9 Conceptual Distinctions: Five Operations **12**
- 9.1 Key generation vs. key derivation . . . 12
- 9.2 Public-key cryptography . . . 12
- 9.3 Hashing . . . 13

9.4 Checksums 13
9.5 Encoding 13

10 Why Multiple Techniques? The Principle of Cryptographic Separation 14

11 Statistical Properties of the Full Pipeline 14
11.1 Uniformity of addresses 14
11.2 Avalanche-like behaviour 15
11.3 Coordinate distribution on secp256k1 15
11.4 Comparison with hash-function statistical properties 15

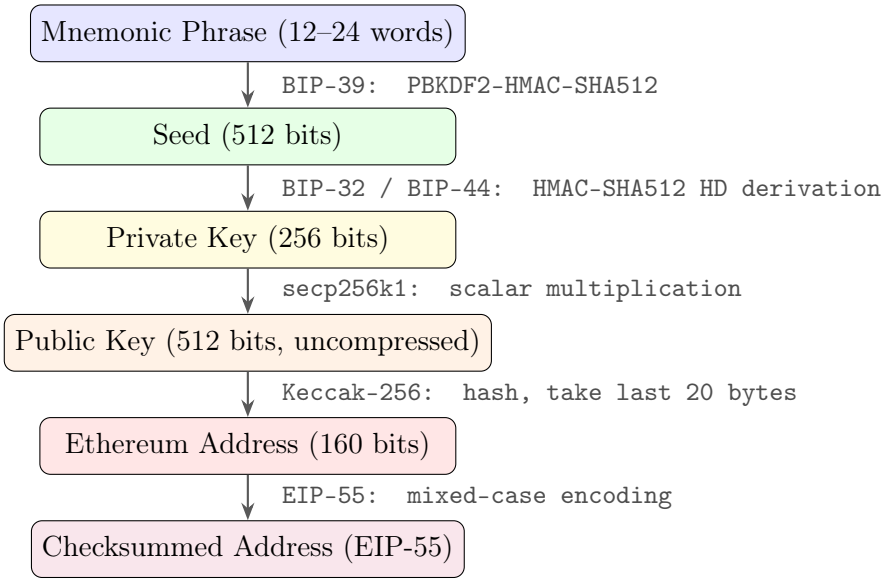
12 Practical Exploration: Metamask and Rabby 16
12.1 Observable steps in Metamask 16
12.2 Cross-wallet verification 16
12.3 A note on hardware wallets 16

13 Summary: The Pipeline as a System 16

1. Overview: The Ethereum Wallet Pipeline

An Ethereum wallet does not store coins directly; it stores and manages *cryptographic key material* that authorises spending. The full pipeline from a human-readable phrase to a checksummed on-chain address involves five distinct steps, each employing a different cryptographic primitive chosen for a specific purpose.

1.1 The pipeline at a glance



1.2 Five fundamental operations – a preliminary taxonomy

Before examining each standard, it is essential to draw clear boundaries between the categories of operation at play. Table 1 summarises them; each will be revisited in detail.

Table 1: Taxonomy of cryptographic operations in the Ethereum pipeline.

Operation	Standards used	Core property
Key generation	OS entropy, BIP-39	Produces fresh secret material from randomness
Key derivation	BIP-32, BIP-44	Deterministically extends one secret into many
Asymmetric crypto	secp256k1	Links a private key to a public key; enables signatures
Hashing	Keccak-256	One-way, fixed-length digest of arbitrary data
Checksum/encoding	EIP-55	Embeds error-detection information in the address string

2. Temporal and Historical Context

Understanding *why* each primitive was chosen requires placing it in the historical moment at which it was introduced.

Table 2: Chronological order of relevant cryptographic standards.

Standard	Year	Significance
RSA	1977	First practical public-key cryptosystem (Rivest, Shamir, Adleman). Security relies on integer factorisation; key sizes are large by modern standards.
secp256k1	1999	Koblitz curve over $\mathbb{F}_p$; defined in SECG standards. Very low adoption until Bitcoin chose it in 2009; then it became the dominant blockchain curve.
SHA-2 / SHA-512	2001	NIST standardisation. SHA-512 is the basis for HMAC-SHA512 used inside BIP-32 derivation. It belongs to the Merkle-Damgård family.
Bitcoin / secp256k1 mainstream	2009	Bitcoin's adoption of secp256k1 brought widespread implementation and auditing.
BIP-32	2012	Hierarchical deterministic wallets (Pieter Wuille). A single root seed generates an entire tree of keys.
BIP-39	2013	Mnemonic codes for HD wallet seeds (Marek Palatinus et al.). Human-memorable backup for key material.
BIP-44	2014	Multi-account HD wallets (Marek Palatinus, Pavol Rusnak). Standardises the derivation path structure across coin types.
SHA-3 (NIST)	2015	Keccak-c[1600] selected in 2012 NIST competition, standardised in 2015. NIST added a different padding (`0x06` vs `0x01`), so SHA3-256 $\neq$ Keccak-256.
EIP-55	2016	Ethereum mixed-case checksum addresses (Vitalik Buterin). Uses Keccak-256 to capitalise address hex digits.

2.1 SHA-2 vs. Keccak-256 / SHA-3: a subtle but important distinction

Both SHA-2 and Keccak (the basis of SHA-3) are cryptographic hash functions, but they differ in design and – crucially – Ethereum does *not* use the NIST-standardised SHA3-256.

SHA-2 (Merkle-Damgård) family

SHA-256 and SHA-512 use a **Merkle-Damgård** construction with a Davies-Meyer compression function. The input is padded and processed in fixed-size blocks; each block updates a running state via a compression function. SHA-512 produces 512-bit digests and is used inside BIP-32's HMAC construction.

Keccak-256 (sponge construction)

Keccak uses a **sponge construction** over a 1600-bit permutation (`Keccak-f[1600]`). There is no length-extension vulnerability inherent to Merkle-Damgård. The sponge absorbs input and then squeezes output. When NIST standardised it as SHA-3 in 2015, they changed the padding from `0x01` to `0x06` for domain separation. Ethereum predates this change and uses the original **Keccak-256** with padding byte `0x01`, which differs from SHA3-256.

Warning for implementers

Never substitute SHA3-256 (the NIST standard) for Keccak-256 in an Ethereum implementation. They produce different digests for the same input.

3. BIP-39 — Mnemonic Sentence and Seed Generation

3.1 Purpose and role

BIP-39 addresses a usability problem: a 256-bit private key is an opaque sequence of bytes, impossible to memorise or safely hand-copy. The proposal translates key material into a short sequence of common English words (*mnemonic phrase* or *seed phrase*) that is easier to write down, verify, and recover from.

3.2 Entropy generation

The process begins with a **cryptographically secure** source of entropy:

1. Generate n bits of entropy from the OS CSPRNG (128, 160, 192, 224, or 256 bits), where a larger n gives more words (12–24).
2. Compute the SHA-256 hash of the entropy; take the first $n/32$ bits as a **checksum**.
3. Concatenate entropy $\|$ checksum $\Rightarrow$ total $n + n/32$ bits.
4. Split into groups of 11 bits; each 11-bit value (0–2047) indexes a word in the BIP-39 wordlist (2048 words).

Example: 128-bit entropy → 12 words

```
Entropy:    128 bits  (generated by OS)
Checksum:     4 bits  (first 4 bits of SHA-256(entropy))
Total:      132 bits = 12 x 11-bit groups -> 12 words
```

```
Example (NOT a real wallet):
"abandon abandon abandon abandon abandon abandon
 abandon abandon abandon abandon abandon about"
```

3.3 Seed derivation: PBKDF2-HMAC-SHA512

The mnemonic phrase is converted to a 512-bit seed via **PBKDF2-HMAC-SHA512**:

$$\text{Seed} = \text{PBKDF2}\big(\text{HMAC-SHA512},\ \underbrace{\textit{mnemonic}}_{\text{password}},\ \underbrace{\text{"}\texttt{mnemonic}\text{"} \,\|\, \textit{passphrase}}_{\text{salt}},\ \underbrace{2048}_{\text{iterations}},\ \underbrace{512}_{\text{bits}}\big)$$

- The **salt** is the string `"mnemonic"` concatenated with an optional user-supplied *passphrase* (the "25th word").
- 2048 iterations provide **key stretching**, raising the cost of brute-force attacks.
- The 512-bit output is the *seed*, which becomes the root of the BIP-32 tree.

3.4 Key properties

- **Deterministic**: the same mnemonic + passphrase always produce the same seed.
- **One-way**: recovering the mnemonic from the seed is computationally infeasible (PBKDF2 is a one-way KDF).
- **Error-detectable**: the internal 4-bit checksum catches single-word transcription errors with high probability.

4. BIP-32 — Hierarchical Deterministic Key Derivation

4.1 Motivation

Before HD wallets, a typical wallet generated a fresh random private key for each transaction. Backing up all keys required exporting every individual key file; losing a backup meant losing funds. BIP-32 introduced a **tree structure** where a single 512-bit seed deterministically generates an unlimited number of key pairs.

4.2 The master node

Given the BIP-39 seed S (512 bits):

$$I = \text{HMAC-SHA512}(\text{key} = \text{"}\texttt{Bitcoin seed}\text{"},\ \text{data} = S)$$

Split I into two 256-bit halves:

$$I_L \,\|\, I_R \quad \Rightarrow \quad k_{\text{master}} = I_L, \quad c_{\text{master}} = I_R$$

where k_{master} is the master private key and c_{master} is the master **chain code** (an extra 256-bit secret that prevents a child key from being derived without knowledge of the parent chain code).

4.3 Child key derivation

For each child index i (0 to $2^{32}-1$):

$$I = \text{HMAC-SHA512}\Big(\text{key} = c_{\text{parent}},\ \text{data} = \begin{cases} \texttt{0x00} \,\|\, k_{\text{par}} \,\|\, \text{ser}_{32}(i) & \text{if } i \geq 2^{31} \text{ (hardened)} \\ \text{ser}_{33}(K_{\text{par}}) \,\|\, \text{ser}_{32}(i) & \text{if } i < 2^{31} \text{ (normal)} \end{cases}\Big)$$

$$k_{\text{child}} = (I_L + k_{\text{parent}}) \bmod n, \qquad c_{\text{child}} = I_R$$

where n is the order of the secp256k1 group.

Normal vs. Hardened derivation

Normal ($i < 2^{31}$): Uses the parent *public* key; allows deriving child public keys without the parent private key (useful for watch-only wallets).

Hardened ($i \geq 2^{31}$, written i' or i_H): Uses the parent *private* key; child public keys cannot be derived from the parent public key alone. Adds security at the cost of flexibility.

4.4 Extended keys and serialisation

A pair (k, c) or (K, c) is called an **extended key** and serialised as a 78-byte structure:

$$\underbrace{4}_{\text{version}}\ \underbrace{1}_{\text{depth}}\ \underbrace{4}_{\text{parent fingerprint}}\ \underbrace{4}_{\text{child index}}\ \underbrace{32}_{\text{chain code}}\ \underbrace{33}_{\text{key}}\ \text{bytes}$$

This is then Base58Check-encoded as `xprv...` (private) or `xpub...` (public).

5. BIP-44 — Standard Derivation Paths for HD Wallets

5.1 The problem: path chaos

BIP-32 defines *how* to derive children but not *which path* to use. Different wallets used incompatible paths, making cross-wallet recovery impossible. BIP-44 standardises the path structure.

5.2 The BIP-44 path

$$m \,/\, 44' \,/\, \text{coin_type}' \,/\, \text{account}' \,/\, \text{change} \,/\, \text{address_index}$$

Table 3: BIP-44 path levels and their semantics.

Level	Hardened?	Meaning
`m`	—	Master node (from BIP-39 seed)
`44'`	Yes	Purpose: signals BIP-44 structure
`coin_type'`	Yes	Coin: 0' = Bitcoin, 60' = Ethereum, etc. (SLIP-44 registry)
`account'`	Yes	Logical user account (0', 1', ...)
`change`	No	0 = external (receiving), 1 = internal (change)
`address_index`	No	Sequential address index (0, 1, 2, ...)

Example: first Ethereum receiving address

```
m / 44' / 60' / 0' / 0 / 0
          ^^    ^^   ^    ^   ^
          |     |    |    |   address index 0
          |     |    |    external chain
          |     |    account 0
          |     Ethereum (coin_type = 60)
          BIP-44 purpose
```

5.3 Why hardened levels matter here

Account, coin type, and purpose are hardened to prevent a leak of one account's extended public key from compromising sibling accounts. The change and address levels are *not* hardened so that a hardware wallet can export `xpub` for the account and a watch-only companion app can derive all receiving addresses without holding the private key.

6. secp256k1 — Elliptic-Curve Public-Key Cryptography

6.1 The curve

secp256k1 is defined over the prime field $\mathbb{F}_p$ with:

$$E : y^2 \equiv x^3 + 7 \pmod{p}$$

$$p = 2^{256} - 2^{32} - 2^9 - 2^8 - 2^7 - 2^6 - 2^4 - 1 = \texttt{FFFFFFFF...FFFFFFFEFFFFFC2F}_{16}$$

Group order (number of points):

$$n = \texttt{FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141}_{16}$$

Base point G (compressed form):

$$G_x = \texttt{79BE667EF9DCBBAC55A06295CE870B07029BFCDB2DCE28D959F2815B16F81798}_{16}$$

6.2 From private key to public key

Given a private key $k \in \{1, \ldots, n-1\}$ (a 256-bit integer chosen uniformly at random), the corresponding public key is:

$$Q = k \cdot G$$

where $\cdot$ denotes **elliptic-curve scalar multiplication**: adding the point G to itself k times using the point-addition law on E.

Computational asymmetry (ECDLP)

Forward (easy): Given k and G, compute $Q = kG$ in $O(\log k)$ steps using double-and-add.

Reverse (hard): Given Q and G, recover k – the *Elliptic Curve Discrete Logarithm Problem* (ECDLP). No sub-exponential algorithm is known for secp256k1; the best attack is $O(\sqrt{n}) \approx 2^{128}$ operations.

6.3 Public key format

The uncompressed public key is the pair (Q_x, Q_y), serialised as:

$$\texttt{04} \,\|\, Q_x \,\|\, Q_y \quad (65 \text{ bytes})$$

For address derivation in Ethereum, the `04` prefix is dropped, leaving the 64-byte concatenation $Q_x \,\|\, Q_y$ (512 bits).

6.4 Why secp256k1 and not RSA?

Table 4: RSA vs. secp256k1 comparison.

	RSA-2048	**secp256k1**
Security level	$\approx$ 112 bits	$\approx$ 128 bits
Key size	2048 bits (private)	256 bits (private)
Hard problem	Integer factorisation	ECDLP
Signature size	256 bytes	64 bytes (ECDSA)
On-chain cost	High (large txns)	Low

ECC achieves higher security per bit; this directly reduces transaction sizes and gas costs on Ethereum.

7. Keccak-256 — Hash Function for Address Derivation

7.1 From public key to address

Given the 64-byte public key $Q_x \,\|\, Q_y$:

$$h = \text{Keccak-256}(Q_x \,\|\, Q_y) \quad (32 \text{ bytes} = 256 \text{ bits})$$

$$\text{address} = h[12:32] \quad (\text{last } 20 \text{ bytes} = 160 \text{ bits})$$

The address is then represented as a 40-character hexadecimal string, prefixed with `0x`.

7.2 Sponge construction

Keccak-256 uses the sponge construction with:

- State width: $b = 1600$ bits (organised as a $5 \times 5 \times 64$ array of lanes).
- Rate: $r = 1088$ bits; capacity: $c = 512$ bits.
- Permutation: Keccak-f[1600], 24 rounds of five steps (θ, ρ, π, χ, ι).
- Output: first 256 bits of the squeezed state.

7.3 Required properties and why they matter

Table 5: Hash function security properties and their wallet-level relevance.

Property	**Formal meaning**	**Wallet consequence**
Pre-image resistance	Given h, infeasible to find x s.t. $H(x) = h$	Cannot recover public key from address
Second pre-image	Given x, infeasible to find $x' \neq x$ s.t. $H(x) = H(x')$	No address collision attack
Collision resistance	Infeasible to find any (x, x') s.t. $H(x) = H(x')$	Birthday-attack resistance
Avalanche effect	Flipping 1 input bit changes $\approx$ 50% of output bits	Statistical uniformity of addresses

7.4 Statistical properties of Keccak-256 outputs

Because Keccak-256 is a **pseudo-random function** (PRF), its outputs over a keyed or varied input space exhibit:

- **Bit uniformity**: each output bit is ≈ 0.5, i.e., approximately half of addresses have any given bit set.
- **Avalanche**: a single-bit change in the public key alters ≈ 128 of the 256 output bits (strict avalanche criterion).
- **No visible correlation**: consecutive private keys $(k, k+1, k+2, \ldots)$ produce addresses that appear independent.

- **Truncation to 160 bits**: taking only the last 20 bytes maintains uniformity; the truncated output is still modelled as a uniform random variable over $\{0,1\}^{160}$.
- **Birthday paradox**: collision probability reaches 50% at approximately 2^{80} addresses, far beyond any practical exhaustion.

Keccak-256 vs. SHA3-256 – reminder

$$\text{Keccak-256}(x) \neq \text{SHA3-256}(x)$$

The difference is in the multi-rate padding. Ethereum uses Keccak-256 (`0x01` padding); NIST SHA-3 uses `0x06` padding. Always verify which variant a library implements before using it in an Ethereum context.

8. EIP-55 — Mixed-Case Checksum Encoding

8.1 The problem

An Ethereum address `0xd3cda913deb6f279d5fba53ab52ff16e35fa5890` is 40 hexadecimal characters (160 bits). Hex is case-insensitive, so mistyping one character goes undetected – a catastrophic error when it means sending funds to the wrong address.

8.2 The EIP-55 algorithm

Vitalik Buterin's EIP-55 (2016) embeds a checksum into the *capitalisation* of the address letters, using Keccak-256 of the lowercase address hex string:

1. Write the address as a 40-character lowercase hex string (no `0x` prefix): $a = a_0 a_1 \ldots a_{39}$.
2. Compute $h = \text{Keccak-256}(a)$ (treating a as an ASCII string). h is 32 bytes = 64 hex nibbles $h_0 h_1 \ldots h_{63}$.
3. For each character a_i:

$$A_i = \begin{cases} \text{uppercase}(a_i) & \text{if } a_i \in \{a, \ldots, f\} \text{ and } h_i \geq 8 \\ a_i & \text{otherwise} \end{cases}$$

4. Prepend `0x`: result is `0x`$A_0 A_1 \ldots A_{39}$.

Example

```
Lowercase:  0xd3cda913deb6f279d5fba53ab52ff16e35fa5890
Keccak hash of "d3cda913deb6f279d5fba53ab52ff16e35fa5890":
            7f2b9a1e...  (each nibble determines case)
EIP-55:     0xd3CDA913dEb6f279D5FBA53Ab52FF16e35FA5890
```

8.3 Error detection capability

A single random character error has a probability of being detected equal to:

$$P(\text{detect}) = 1 - \frac{1}{16} \approx 93.75\%$$

for each changed letter digit. Over a full 40-character address (26 letter positions), essentially any single-character mutation is caught with very high probability. EIP-55 does *not* correct errors; it only *detects* them.

8.4 Backward compatibility

A purely numeric address (`0x1234...`) has no letter digits; EIP-55 encoding is a no-op for it. Wallets can safely accept both mixed-case (checksummed) and all-lowercase (unchecksummed) addresses.

9. Conceptual Distinctions: Five Operations

This section draws the boundaries that the homework explicitly requires.

9.1 Key generation vs. key derivation

Key generation

Key generation produces fresh secret material from an independent source of randomness (entropy). Each call to a key-generation routine produces a statistically independent key. In the Ethereum pipeline, key generation occurs *once*: the OS provides 128–256 bits of entropy to BIP-39 to form the mnemonic.

Key derivation

Key derivation deterministically extends *existing* secret material into new key material without consuming fresh entropy. BIP-32 and BIP-44 are key-derivation mechanisms: given the same root seed, they always produce the same tree of keys. The security guarantee is not "every key is random" but "knowledge of the root implies knowledge of all derived keys, and nothing short of that does".

Why the distinction matters. Losing a BIP-32 seed means losing the *entire tree*; there is no independent backup for each address. Conversely, a single secure backup of the mnemonic recovers every address on every chain.

9.2 Public-key cryptography

secp256k1 is the pipeline's **asymmetric** or public-key component. Its purpose is to create a **mathematically linked key pair** (k, Q) such that:

- Q can be published safely (the *public* key / Ethereum address is derived from it).
- Only the holder of k can produce a valid ECDSA signature authorising a transaction.

- The relationship $Q = kG$ is a one-way function (ECDLP hardness).

Public-key cryptography is the foundation of ownership on Ethereum: funds locked to address A can be spent only by the entity that knows the private key corresponding to A.

9.3 Hashing

Keccak-256 is the pipeline's **hash** step. A hash function maps arbitrary-length input to a fixed-length output and must be:

- Pre-image resistant (one-way),
- Collision resistant,
- Fast to compute (unlike PBKDF2's intentional slowness).

In the pipeline, hashing serves two roles:

1. **Compressing the public key**: 512 bits → 160 bits (the address).
2. **Providing an additional layer of hiding**: even if the ECDLP were weakened, an attacker would still need to invert Keccak-256 to go from address to public key before trying to recover the private key.

9.4 Checksums

EIP-55 is a **checksum** mechanism. A checksum does not add security or secrecy; it adds **error-detection** capability. The checksummed address encodes redundant information (the Keccak-256 of itself) in a way that allows a receiver to verify that the address was transcribed correctly.

Checksums protect against *accidental* errors (typos, copy-paste mistakes, OCR errors). They do not protect against deliberate forgery (an attacker can compute a valid EIP-55 checksum for any address).

9.5 Encoding

Encoding is a **representation transformation** that carries no cryptographic security property. Examples in the pipeline:

- Hex encoding: converts raw bytes to printable characters.
- Base58Check: encodes extended keys (`xprv`/`xpub`).
- BIP-39 words: encodes 11-bit chunks as English words.
- EIP-55 mixed case: encodes the checksum within the capitalisation of the hex string.

Encoding is reversible and lossless by design. It exists for **interoperability and human usability**, not for security.

10. Why Multiple Techniques? The Principle of Cryptographic Separation

A naive design might ask: why not use a single algorithm for everything? The answer is that the five operations serve **fundamentally different purposes**, and no known primitive satisfies all of them simultaneously.

Table 6: Purpose separation in the Ethereum pipeline.

Primitive(s)	**Category**	**Specific purpose**
BIP-39	KDF + encoding	Convert entropy to human-memorable form; one-way seed stretching
BIP-32/44	Deterministic KDF	Derive a tree of keys from one root without fresh randomness
secp256k1	Asymmetric crypto	Create a private/public key pair; enable digital signatures
Keccak-256	Hash function	Irreversibly compress public key to address; avalanche mixing
EIP-55	Checksum/encoding	Detect transcription errors in address strings

Additional reasons for specialisation.

1. **Security assumptions are independent.** ECDLP hardness says nothing about hash pre-image resistance. If secp256k1 were broken by a quantum algorithm (Shor's), Keccak-256 would still protect the address against direct reconstruction.
2. **Performance profile.** PBKDF2 must be *slow* (key stretching); Keccak-256 must be *fast* (used in every state transition in the EVM). These requirements are contradictory.
3. **Standardisation and auditability.** Each component has an independent specification, independent implementations, and independent security proofs or cryptanalysis. Combining them into one algorithm would require re-auditing the combined system.
4. **Modularity for future migration.** If one primitive is weakened, only that layer needs replacement. The rest of the pipeline can remain unchanged.

11. Statistical Properties of the Full Pipeline

11.1 Uniformity of addresses

An Ethereum address is the Keccak-256 digest of a secp256k1 public key, truncated to 160 bits. Because:

1. Private keys are sampled uniformly from $\{1, \ldots, n-1\}$,
2. The map $k \mapsto kG$ is a pseudorandom injection (ECDLP hardness),
3. Keccak-256 behaves as a PRF over its input space,

addresses are indistinguishable from uniform random elements of $\{0,1\}^{160}$.

11.2 Avalanche-like behaviour

Changing a single bit of the private key:

- Produces a completely different public key (scalar multiplication is highly non-linear over the curve).
- Further scrambled by Keccak-256: approximately half of the 256 output bits change.
- Results in a totally different address.

This is analogous to – but mechanistically different from – the strict avalanche criterion of a hash function. The non-linearity here is geometric (elliptic curve) rather than bitwise.

11.3 Coordinate distribution on secp256k1

For random private keys k, the x- and y-coordinates of $Q = kG$:

- Are distributed roughly uniformly over $\mathbb{F}_p$ (proved under the ECDLP hardness assumption).
- Show no detectable correlation between Q_x and Q_y for independent k.
- Are not exactly uniform (the curve has n points, $n < p$, so approximately $1/p$ values of x have no corresponding point).

11.4 Comparison with hash-function statistical properties

Table 7: Statistical properties: cryptographic hash vs. EC scalar multiplication.

Property	**Keccak-256 (hash)**	**secp256k1 ($k \mapsto kG$)**
Output looks random	Yes (PRF)	Yes (under ECDLP)
Strict avalanche criterion	Yes ($\approx 50\%$ bit flip)	Approximately yes
Collision resistance	Yes (2^{128} effort)	N/A (injective)
Pre-image resistance	Yes	Yes (ECDLP)
Fixed output size	Yes (256 bits)	No (points in $\mathbb{F}_p^2$)
Second pre-image	Yes	N/A
Based on algebraic structure	No (purely bitwise/byte)	Yes (group law)
Quantum vulnerability	Grover $O(2^{128})$	Shor $O(\text{poly})$

Key distinction. A cryptographic hash function is a *symmetric* primitive: there is no structural relationship between inputs and outputs beyond "same input gives same output." Elliptic-curve scalar multiplication is an *asymmetric* primitive: the group structure links k and kG algebraically, enabling digital signatures (ECDSA) that are impossible with a hash alone.

12. Practical Exploration: Metamask and Rabby

Wallets such as Metamask, Rabby, Exodus, and Rainbow implement the full BIP-39/32/44 pipeline. Observing them in practice helps connect theory to implementation.

12.1 Observable steps in Metamask

1. **Wallet creation**: the browser extension generates 128 bits of entropy and presents 12 BIP-39 words.
2. **Account derivation**: the first account uses path `m/44'/60'/0'/0/0`. Adding a second account increments the last index to `m/44'/60'/0'/0/1`.
3. **Address display**: addresses are shown in EIP-55 mixed-case format.
4. **Seed phrase recovery**: entering the same 12 words in a new wallet instance recovers the same addresses – demonstrating BIP-39/32 determinism.

12.2 Cross-wallet verification

The same BIP-39 mnemonic entered in Metamask, Rabby, and Exodus produces identical Ethereum addresses at each BIP-44 path position. This is only possible because all three implement the same open standards without proprietary modifications. Verifying this equivalence with multiple tools is the best practical evidence of the pipeline's correctness.

12.3 A note on hardware wallets

Ledger and Trezor devices implement the same pipeline but perform all private key operations inside a secure element. The BIP-39 mnemonic is generated on-device; the private key never leaves the device. The host computer receives only public keys and signed transactions – a direct demonstration of why the public key step is safe to expose.

13. Summary: The Pipeline as a System

The complete wallet pipeline

1. **Entropy** (OS): 128–256 bits of true randomness.
2. **BIP-39**: entropy → mnemonic (encoding + checksum) → seed (PBKDF2-HMAC-SHA512, 2048 iterations). Purpose: usability, one-way stretching.
3. **BIP-32**: seed → master node; child nodes via HMAC-SHA512 on the parent

chain code. Purpose: deterministic tree.

4. **BIP-44**: specifies which tree path to use (`m/44'/60'/0'/0/`i). Purpose: interoperability.
5. **secp256k1**: private key $k \to$ public key $Q = kG$. Purpose: asymmetric cryptography, ownership, signatures.
6. **Keccak-256**: public key $\to$ 256-bit hash; take last 20 bytes. Purpose: address compression, extra one-way layer.
7. **EIP-55**: lowercase hex $\to$ mixed-case hex. Purpose: transcription error detection.

Each step is necessary; none is redundant. Together they form a layered security architecture in which each layer's specific guarantee compensates for what the others cannot provide.